Tiny Camel: Real-Time TinyML Autonomy on ESP32-S3

Harry Wang
University of Kansas
Lawrence, KS, USA

Abdulaziz Ali
University of Kansas
Lawrence, KS, USA

Isaac Mohabbat
University of Kansas
Lawrence, KS, USA

Abstract

This paper presents Tiny Camel, a replication and extension of the DeepPicarMicro platform that demonstrates real-time autonomous control using TinyML on the ESP32-S3 microcontroller. Building upon prior work in end-to-end deep learning for autonomous vehicles, we investigate the accuracy-latency trade-offs in resource-constrained cyber-physical systems using a more capable MCU with AI acceleration. We implement two distinct vehicle architectures—a traditional front-wheel steering RC car and a differential-drive system using modified servo motors—to study control challenges in TinyML deployment. Using a simplified single-lane track and MobileNetV2-based CNN, we demonstrate autonomous navigation while identifying critical challenges in data quality, motor control granularity, and real-time performance. Our experimental results reveal inference latencies of approximately 800ms and highlight the fundamental tension between model complexity, control responsiveness, and dataset quality in MCU-based autonomous systems.

1 Introduction

The rapid advancement of deep neural networks (DNNs) has revolutionized autonomous vehicle perception and control, enabling end-to-end learning approaches that directly map sensor inputs to control commands [1]. However, deploying these computationally intensive models on resource-constrained embedded systems remains a significant challenge, particularly for real-time cyber-physical systems (CPS) where inference latency directly impacts control stability and safety.

Recent work in TinyML—the execution of machine learning models on microcontroller units (MCUs)—has demonstrated the feasibility of running neural networks on severely resource-constrained devices [3]. The DeepPicarMicro platform showed that end-to-end autonomous control could be achieved on a Raspberry Pi Pico (RP2040) MCU, revealing that faster, less accurate models often outperform slower, more accurate ones in closed-loop control scenarios [3]. This insight challenges conventional ML optimization practices and suggests that control-aware model selection is essential for autonomous CPS.

In this work, we present Tiny Camel, a replication and extension of DeepPicarMicro using the Seeed Studio ESP32-S3 microcontroller. Our platform offers several advantages over the original RP2040-based system: dual-core Xtensa processors running at 240 MHz, dedicated vector AI acceleration instructions, increased RAM with optional PSRAM, and support for higher frame-rate cameras. These enhancements enable us to explore tighter control deadlines and investigate whether additional computational resources fundamentally alter the accuracy-latency trade-offs observed in prior work.

We make the following contributions:

- A working TinyML-based autonomous vehicle platform on ESP32-S3 with two distinct mechanical architectures for comparative analysis
- Experimental validation of end-to-end CNN control on a simplified single-lane track, revealing unique challenges in data collection and model training
- Comprehensive latency measurements demonstrating inference times of approximately 800ms and identification of blocking operations as critical bottlenecks
- Analysis of how mechanical constraints (steering granularity, speed-dependent turning radius) interact with ML model performance in real-world deployment

Our work demonstrates that while more powerful MCUs enable higher frame rates and more complex models, fundamental challenges in data quality, motor control precision, and real-time determinism remain central to successful TinyML deployment in autonomous CPS.

2 Background and Related Work

2.1 End-to-End Learning for Autonomous Vehicles

End-to-end deep learning approaches eliminate traditional modular pipelines (perception, planning, control) by training a single neural network to directly map sensor inputs to control outputs. NVIDIA's PilotNet demonstrated this approach at scale, using a convolutional neural network to predict steering commands from camera images [1]. The system emphasized data quality over quantity, introducing tools to identify problematic training samples and demonstrating that networks can exploit false correlations in augmented data rather than learning true lane-center semantics [1].

DeepPicar brought this paradigm to low-cost embedded platforms, implementing the DAVE-2 architecture on a $70 Raspberry Pi 3-based RC car [2]. The work revealed that CNN inference dominates control loop execution time (approximately 85%) and identified memory bandwidth contention as a critical bottleneck for real-time DNN-based control on multicore embedded platforms [2].

2.2 TinyML and Microcontroller-Based Autonomy

DeepPicarMicro extended this research to microcontroller-class hardware, implementing end-to-end autonomous control on a Raspberry Pi Pico (RP2040) MCU [3]. Through systematic optimization including 8-bit quantization, depthwise separable convolutions, and neural architecture search (NAS), the work achieved real-time inference within a 133ms control deadline. Critically, the study revealed that lower-latency models often outperform more accurate but slower models in real CPS, proposing a joint accuracy-latency optimization heuristic:

$$\text{Score} = \text{normalized(validation loss)} + \text{normalized(latency)} \quad (1)$$

This insight fundamentally challenges the conventional practice of treating latency as a hard constraint while optimizing solely for accuracy [3].

2.3 Embedded Systems for Autonomous Vehicles

Recent work has explored various embedded platforms for autonomous navigation. GPS-based systems using ESP32 microcontrollers have demonstrated feasibility for waypoint navigation and obstacle avoidance [4], while camera-based parking assistance systems have shown that combining ultrasonic sensors with embedded ML on ESP32-CAM enables context-aware spatial reasoning [5]. These systems highlight the importance of sensor fusion and the challenges of deploying vision-based AI on resource-constrained hardware.

Our work builds upon these foundations by implementing end-to-end learning on the more capable ESP32-S3 platform, investigating whether increased computational resources fundamentally alter the trade-offs observed in prior TinyML autonomous systems.

3 System Design

3.1 Hardware Platform

We implemented two distinct vehicle architectures to study different control paradigms and mechanical constraints:

3.1.1 Traditional RC Car. The traditional configuration uses a commercial RC car chassis with front-wheel steering and rear-wheel drive. This architecture mirrors conventional vehicle dynamics but introduces speed-dependent steering behavior, where higher velocities result in larger turning radii. The system comprises:

- Seeed Studio ESP32-S3 (dual-core Xtensa @ 240 MHz)
- OV2640/OV5640 camera module
- L298N H-bridge motor driver
- Commercial RC car chassis from Walmart
- 7.4V LiPo battery pack

3.1.2 Differential-Drive Servo Car. To address control granularity limitations, we developed a custom differential-drive platform using modified continuous-rotation servo motors. This design offers finer motion control but requires additional electronics:

- Seeed Studio ESP32-S3 (dual-core Xtensa @ 240 MHz)
- Arduino Uno as voltage converter (3.3V to 5V) and servo command parser
- Two modified continuous-rotation servo motors with integrated gearboxes
- Custom chassis with differential steering

The ESP32-S3 provides significant advantages over the RP2040 used in DeepPicarMicro [3]: vector AI acceleration instructions for faster inference, 512KB SRAM with optional PSRAM, and native support for higher frame-rate cameras enabling tighter control deadlines.

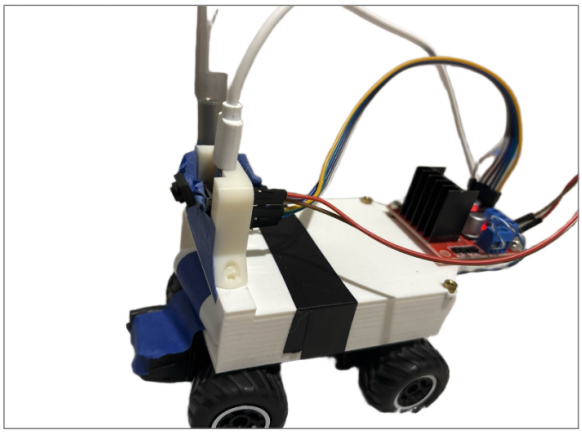


Figure 1: The custom differential-drive servo car chassis developed to address steering granularity issues.

3.2 Track Design and Simplifications

Unlike DeepPicarMicro's two-lane bordered track [3], we implemented a simplified single-lane track marked with blue tape. This design decision was motivated by home replicability and reduced dataset complexity. However, this simplification introduced an unforeseen challenge: if the vehicle loses track of the single lane, it cannot recover autonomously, as there are no boundary cues to guide it back onto the path.

The single-lane approach transforms the task into a camera-based line follower, reducing the perceptual complexity but increasing the precision requirements for continuous tracking.

3.3 Software Architecture

3.3.1 Control Pipeline. Our control loop implements the following stages:

(1) **Frame Capture**: Acquire image from camera module
(2) **Preprocessing**: Crop, convert to grayscale, resize, apply color masking for blue tape isolation
(3) **CNN Inference**: Forward pass through quantized MobileNetV2
(4) **Decision Logic**: Classify output probabilities into discrete actions (left-forward, forward, right-forward)
(5) **Motor Control**: Send binary on/off commands to motor driver

Actions are triggered when output probabilities exceed a threshold of 0.80, ensuring high-confidence decisions. The entire pipeline must complete before the next frame arrives to maintain real-time responsiveness.

3.3.2 Real-Time Constraints. The fundamental timing constraint for autonomous control is:

$$T_{\text{react}} = T_{\text{capture}} + T_{\text{preprocess}} + T_{\text{inference}} + T_{\text{actuation}} \quad (2)$$

This total reaction time must satisfy:

$$T_{\text{react}} \leq \alpha \cdot \frac{d_{\text{visible}}}{v_{\text{vehicle}}} \quad (3)$$



Figure 2: The simplified single-lane track used for training and testing, marked with blue tape.

where α is a safety margin, d_{visible} is the look-ahead distance, and v_{vehicle} is the vehicle velocity. For our target frame rate of 20–30 FPS, inference must complete within 33–50ms. This represents a significantly tighter deadline than DeepPicarMicro's 133ms control period [3].

3.4 Neural Network Architecture

We employed a MobileNetV2-based architecture optimized for the ESP32-S3 platform. The network uses depthwise separable convolutions to reduce computational complexity while maintaining representational capacity. The architecture consists of:

- Input: 96×96 grayscale images
- MobileNetV2 feature extractor (reduced width multiplier)
- Three fully connected dense layers for classification
- Output: 3-way softmax (left-forward, forward, right-forward)

The model was trained and transformed into a full Int8 quantization using Post Training Quantization (PTO) via TensorFlow Lite Micro (TFLM). This quantization reduces memory footprint and improves inference speed while minimizing accuracy degradation, following best practices established in prior TinyML work [3].

4 Implementation Methodology

4.1 Data Collection

Training data was collected by manually driving the RC car around the track while recording synchronized camera images and steering

Layer (type)	Output Shape	Param #
mobilenetv2_0.35_96 (Functional)	(None, 3, 3, 1280)	410,208
global_average_pooling2d_22 (GlobalAveragePooling2D)	(None, 1280)	0
dense_23 (Dense)	(None, 256)	327,936
batch_normalization_2 (BatchNormalization)	(None, 256)	1,024
dropout_2 (Dropout)	(None, 256)	0
dense_24 (Dense)	(None, 128)	32,896
batch_normalization_3 (BatchNormalization)	(None, 128)	512
dropout_3 (Dropout)	(None, 128)	0
dense_25 (Dense)	(None, 3)	387

Figure 3: The MobileNetV2 architecture adapted for the ESP32-S3 with a custom classification head.

commands. This imitation learning approach mirrors the methodology used in PilotNet [1] and DeepPicar [2]. However, we encountered significant challenges:

4.1.1 Speed-Dependent Steering. The traditional RC car exhibits velocity-dependent turning behavior, where faster speeds produce larger turning radii. This mechanical characteristic complicates data collection, as the relationship between steering input and vehicle trajectory varies with speed. Driving consistency becomes difficult, and poor driving performance directly translates to poor model performance, echoing the "garbage in, garbage out" principle emphasized in PilotNet [1].

4.1.2 Driver Variability. Inconsistent human driving resulted in biased training data. Our confusion matrix analysis revealed a strong tendency toward forward predictions, with significantly fewer examples of left turns compared to right turns. This imbalance reflects the challenges of manual data collection on a physically difficult-to-control platform.

4.1.3 Single-Lane Recovery Problem. Unlike dual-lane tracks that provide boundary cues [3], our single-lane design offers no recovery mechanism. Once the vehicle deviates from the tape, the camera observes only uniform background, making it impossible for the model to determine the correct direction to return to the track.

4.2 Data Preprocessing

To isolate the blue tape from the background, we implemented a color-based masking pipeline:

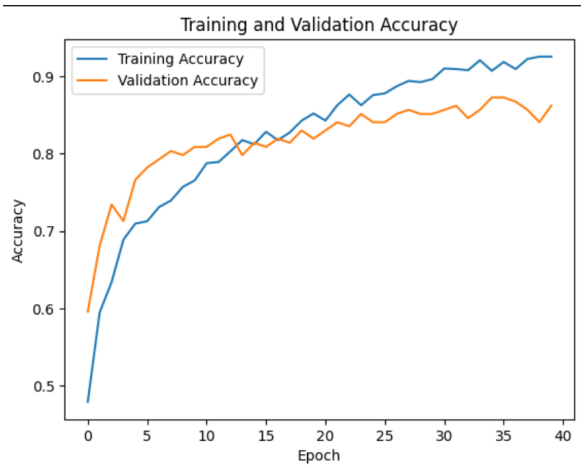
(1) Convert RGB image to grayscale
(2) Apply adaptive thresholding to enhance contrast
(3) Use HSV color space filtering to isolate blue regions
(4) Morphological operations to reduce noise
(5) Resize to 96×96 for neural network input

Despite these efforts, lighting variation and tape visibility remained challenging, consistent with findings in embedded vision systems [5].

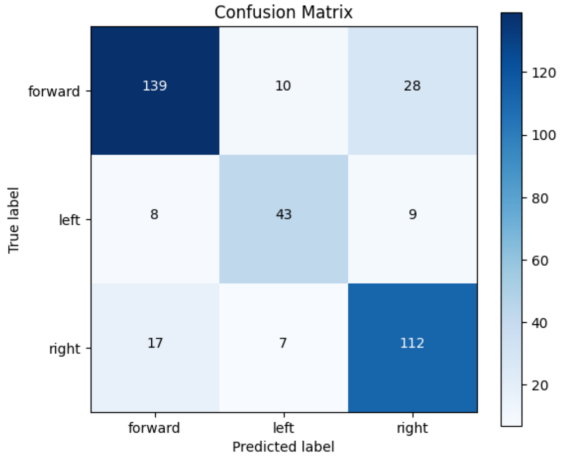
4.3 Model Training and Deployment

Models were trained offline on a desktop workstation with GPU acceleration using TensorFlow. We employed data augmentation including horizontal flips, brightness adjustments, and small rotations to improve generalization. However, following insights from PilotNet [1], we were cautious about introducing artifacts that the network might exploit rather than learning true lane-following semantics.

After training, models were converted to TensorFlow Lite format with Int8 quantization and deployed to the ESP32-S3 via TFLM. The TFLM arena was configured based on model memory requirements, determined through activation analysis similar to the approach in DeepPicarMicro [3]. Training Resulted in approximately 87% validation accuracy, 86% test accuracy, and 67% fully quantized accuracy.



(a) Training and Validation Accuracy



(b) Confusion Matrix

Figure 4: Model training results showing (a) accuracy convergence over epochs and (b) class imbalance in prediction.

5 Experimental Results

5.1 Performance Metrics

We evaluated system performance using the following metrics:

- **Inference Latency**: Time required for CNN forward pass
- **Total Loop Time**: End-to-end control cycle duration
- **Track Completion**: Distance traveled before deviation
- **Confusion Matrix**: Classification accuracy per action class

5.2 Latency Analysis

Our measurements revealed significant timing challenges:

5.2.1 Initial Implementation. In our first iteration, the total control loop time was approximately 1.5 seconds, far exceeding our target deadline of 33–50ms. This catastrophic latency was primarily due to two factors:

(1) MobileNetV2 inference taking approximately 800ms
(2) Blocking delay functions in motor control code

The use of Arduino's `delay()` function, which blocks execution, violated real-time principles. This should have been replaced with non-blocking alternatives such as FreeRTOS `vTaskDelay()` to maintain system responsiveness.

5.2.2 Revised Implementation. After identifying the blocking operations, we estimated that model inference alone consumed approximately 800ms per frame. While this represents a significant improvement over the total 1.5-second loop time, it remains substantially slower than the 33–50ms target and even exceeds DeepPicarMicro's 133ms control period [3].

This finding was unexpected given the ESP32-S3's superior computational capabilities compared to the RP2040. We hypothesize that TFLM optimization for the ESP32-S3's vector instructions may not be as mature as for ARM Cortex-M series processors, or that MobileNetV2's architecture is poorly suited to the ESP32-S3's execution units.

5.3 Autonomous Driving Performance

5.3.1 Attempt 1: High-Speed Operation. Our initial autonomous runs used higher vehicle speeds to maintain motor responsiveness. However, the combination of high velocity and 800ms+ inference latency resulted in severe control lag. By the time the model processed a frame and issued a command, the vehicle had already moved significantly past the observed scene. This staleness of control inputs, combined with the speed-dependent turning radius of the RC car, caused the vehicle to frequently overshoot turns and leave the track.

Once off the single-lane track, the vehicle's behavior became essentially random, as the camera no longer observed the blue tape and the model had no basis for recovery decisions.

5.3.2 Attempt 2: Low-Speed "Stop and Think". To address the latency-velocity mismatch, we implemented a "stop and think" strategy where the vehicle would pause, perform inference, execute the predicted action, then repeat. This approach aimed to eliminate the control lag problem by ensuring commands were based on current rather than stale observations.

However, this strategy introduced new problems:

(1) Motors stalled at very low speeds due to insufficient torque
(2) Blocking delays caused extended periods of inactivity
(3) Jerky motion degraded control quality

The stop-and-go behavior violated the continuous control assumption underlying end-to-end learning, as the vehicle's motion profile during data collection (smooth human driving) differed drastically from deployment (intermittent movement).

5.4 Model Analysis

Confusion matrix analysis of our trained model revealed significant class imbalance effects:

- **Forward**: Strong prediction tendency (high recall)
- **Left-forward**: Very low recall, frequent misclassification
- **Right-forward**: Moderate performance

This imbalance directly reflects the training data distribution, where the difficulty of consistently executing left turns during manual driving resulted in insufficient left-turn examples. The model essentially learned to be "optimistic" about going straight or right, consistent with the distribution of training data rather than the geometric requirements of the track.

This finding reinforces PilotNet's emphasis on data quality [1]: even sophisticated neural architectures cannot overcome fundamentally biased or incomplete training datasets.

6 Discussion

6.1 Latency-Accuracy Trade-offs

Our results provide an interesting counterpoint to DeepPicarMicro's findings [3]. While the ESP32-S3 offers substantially more computational power than the RP2040, we did not observe proportionally faster inference times. The 800ms latency for MobileNetV2 significantly exceeds what would be expected from the clock speed and architectural improvements.

Several factors may explain this discrepancy:

(1) **TFLM Optimization Maturity**: TensorFlow Lite Micro may not fully exploit the ESP32-S3's vector acceleration instructions
(2) **Architecture Mismatch**: MobileNetV2's depthwise separable convolutions may not map efficiently to the Xtensa instruction set
(3) **Memory Bandwidth**: Despite increased RAM, memory access patterns may still bottleneck performance
(4) **Thermal Throttling**: The ESP32-S3 may reduce clock speed under sustained computational load

These findings suggest that simply moving to more powerful MCUs does not automatically solve the latency challenges in TinyML deployment. Architecture-aware model design remains critical.

6.2 Mechanical Constraints and Control

The traditional RC car's speed-dependent steering and lack of fine-grained control significantly complicated the learning task. Unlike simulation environments where control inputs map linearly to vehicle dynamics, our physical platform exhibited complex nonlinear behavior that varied with velocity, battery charge state, and surface friction.

The differential-drive servo car design was intended to address these limitations by providing independent control of left and right wheels. However, the additional complexity of the Arduino-based voltage conversion and command parsing system introduced new latency and reliability concerns that we did not fully resolve.

Future work should prioritize mechanical platforms with linear, predictable control characteristics, as emphasized in DeepPicar's analysis of motor control timing [2].

6.3 Data Quality and Collection Challenges

Our experience strongly validates PilotNet's focus on data quality over quantity [1]. The difficulty of manually driving the traditional RC car, combined with the speed-dependent steering behavior, resulted in training data that was neither diverse nor representative of optimal driving behavior.

Moreover, the single-lane track design eliminated the possibility of learning recovery behaviors, as the model never observed examples of successfully returning to the track after deviations. This is analogous to the "distribution shift" problem in imitation learning, where the agent encounters states during deployment that were never seen during training.

Improved data collection strategies might include:

- Using a more controllable vehicle platform for initial data gathering
- Implementing semi-autonomous data augmentation to generate recovery scenarios
- Employing human-labeled lane centers rather than raw trajectories [1]
- Stratified sampling to ensure balanced representation of all action classes

6.4 Real-Time Determinism

The use of blocking delay functions in our initial implementation violated fundamental real-time systems principles. This highlights an important lesson: TinyML deployment requires not just ML expertise but also embedded systems engineering discipline. Non-blocking designs, RTOS task scheduling, and careful timing analysis are as critical as model optimization.

DeepPicar's detailed timing analysis [2] and DeepPicarMicro's focus on deterministic execution [3] provide valuable frameworks that we should have followed more rigorously from the outset.

7 Challenges and Limitations

7.1 Hardware Challenges

- **Camera Stability**: Achieving consistent frame rates while maintaining image quality proved difficult, especially under varying lighting
- **Motor Control**: The L298N driver and RC car motors lacked fine-grained speed control, resulting in binary "fast" or "stalled" behavior
- **Power Management**: Battery voltage variations affected motor performance and created additional variability in vehicle behavior

7.2 Software Challenges

- **TFLM Arena Sizing**: Determining optimal memory allocation required iterative testing and was model-dependent
- **Preprocessing Overhead**: Color masking and image transformations added latency beyond pure CNN inference
- **Inference Optimization**: Lack of documentation on ESP32-S3-specific TFLM optimizations limited our ability to improve performance

7.3 Methodological Limitations

- **Small Team**: As a three-person team, we had limited time for extensive hyperparameter tuning and architecture search
- **Single Track**: Testing on only one track configuration limited our ability to assess generalization
- **No Simulation**: Lack of a simulation environment prevented rapid iteration and systematic latency studies as in DeepPicarMicro [3]

8 Future Work

Based on our experiences, we propose the following directions for future research:

8.1 Hardware Improvements

- **Higher-Torque Motors**: Implement gear-reduced motors capable of consistent low-speed operation
- **Improved Steering**: Add continuous servo steering or stepper-motor-based control for fine-grained turning
- **Sensor Fusion**: Integrate wheel encoders or IMU data to provide additional control feedback

8.2 Data and Training

- **Expanded Dataset**: Collect 5–10× more training examples with deliberate attention to class balance
- **Better Preprocessing**: Develop more robust blue tape isolation algorithms less sensitive to lighting
- **Transfer Learning**: Explore pre-training on simulation data before fine-tuning on physical platform

8.3 Model Optimization

- **Architecture Search**: Systematically evaluate PilotNet [1], custom CNNs, and other architectures optimized for ESP32-S3
- **Quantization Strategies**: Investigate mixed-precision quantization and other compression techniques
- **Pruning**: Apply structured pruning to reduce inference latency while maintaining accuracy

8.4 System Integration

- **RTOS Implementation**: Properly implement FreeRTOS task scheduling with non-blocking operations
- **Profiling Tools**: Deploy systematic timing measurement infrastructure to identify bottlenecks
- **Continuous Integration**: Establish automated testing pipeline for model deployment validation

9 Conclusion

Tiny Camel demonstrates both the promise and challenges of deploying TinyML-based autonomous control on modern microcontrollers. While the ESP32-S3 offers significant computational advantages over previous MCU platforms, our experience reveals that raw processing power alone does not guarantee real-time performance. Architecture-aware model design, careful systems engineering, and rigorous attention to data quality remain essential.

Our 800ms inference latency, while disappointing relative to our target, provides valuable insights into the optimization challenges facing TinyML on non-ARM architectures. The fundamental tension between model complexity, inference speed, and control quality observed in DeepPicarMicro [3] persists even on more capable hardware, suggesting that this trade-off is intrinsic to resource-constrained autonomous systems rather than merely a hardware limitation.

The mechanical challenges we encountered—speed-dependent steering, limited control granularity, and motor stall behavior—underscore that successful TinyML deployment requires co-design of hardware, software, and learning systems. Neither sophisticated neural networks nor powerful microcontrollers can overcome fundamental mismatches between the physical platform and the control task.

Despite these challenges, our work contributes to the growing body of evidence that microcontroller-based autonomous systems are feasible and educationally valuable. The accessibility of platforms like ESP32-S3 and the continued maturation of TinyML tools promise to make sophisticated autonomous systems increasingly available for research, education, and low-cost applications.

Future iterations of Tiny Camel will benefit from the lessons learned regarding data quality, mechanical design, and real-time systems principles. With systematic improvements to hardware, expanded training datasets, and optimized model architectures, we believe that sub-100ms inference with accurate autonomous control on ESP32-S3 is achievable.

Acknowledgments

We thank the authors of DeepPicarMicro for their comprehensive documentation and open-source implementations, which provided an invaluable foundation for this work.

References

[1] M. Bojarski et al., "End-to-End Deep Learning for Self-Driving Cars," NVIDIA Technical Report, arXiv:2010.08776v1, 2020.

[2] B. C. Jung et al., "DeepPicar: A Low-Cost Deep Neural Network–Based Autonomous Car," *Proceedings of the IEEE International Conference on Embedded and Real-Time Computing Systems and Applications*, 2019.

[3] S. Alyamkin et al., "DeepPicarMicro: Applying TinyML to Autonomous Cyber-Physical Systems," arXiv:2208.11212, 2022.

[4] R. Kumar et al., "Development of Autonomous Vehicle System using ESP32 Microcontroller," *International Journal of Intelligent Transportation Systems Research*, 2023.

[5] S. Sankareh, "Smart Parking Detection System using Ultrasonic Sensors and ESP32-CAM," Master's Thesis, University of Bridgeport, 2024.